Around the World in Eighty Algorithms: Optimising Idealised Global Circumnavigation

Kieran Borovac

Abstract

Pathfinding algorithms are an area of much practical utility and relevance to everyday life. Here we ignore all such applications, and instead try to computationally solve for the fastest possible way to circumnavigate the Earth entirely on commercial flights (in compliance with the rules of the Fédération Aéronautique Internationale).

1 Introduction

A *circumnavigation* is defined as a sequence of scheduled commercial passenger flights comprising a single continuous route, satisfying the following criteria:

1. The first flight departs from some specified *home airport*, and the last flight arrives at the same airport;
2. The route crosses all lines of longitude;
3. The route covers a distance of at least 36,770 km (measured as great arcs between each pair of airports);
4. No airport is visited more than once, except when we return to the home airport at the end of the run.

The above rules are derived from Rule 4.6.4.2 and 4.6.4.3 of Section 2 of the 2016 FAI Sporting Code. [1]

We will additionally make the following highly unrealistic simplifying assumptions:

1. Every flight departs and arrives precisely on schedule;
2. We can transfer between any pair of flights within thirty minutes;
3. Transferring between airports on the ground is not permitted;
4. Concerns relating to food, sleep, cost, safety, border controls, active geopolitical conflicts, and sanity may be ignored.

Inspired by the great airline explorers such as Springbett in 1980 [2] and Chase and Doyle in 2022 [3], I attempt to find an algorithm capable of solving the following problem: **given a starting date, time and home airport, find a circumnavigation that finishes as early as possible.**

2 Notes on Data Sourcing

There is a considerable risk that the software I have developed to solve this frivolous problem may be incidentally useful for finding actual desirable flight routes. This risk has been mitigated by the wisdom and foresight of OAG Aviation Worldwide Ltd., which operates the single most comprehensive repository of flight schedule data in the world; when I contacted OAG to request access to their dataset, they refused to give me a price quote and instead signed me up to their newsletter without asking. Consequently, I have had to fall back on an alternative data source: a 3 GB .csv file which I downloaded from a webpage that was clearly meant to have been password-protected, containing an apparently-accurate copy of OAG's schedule dataset for the calendar year 2022. This means that my software is only capable of producing routes that are several years out of date, and is thus entirely useless for future route planning unless one possesses a time machine.[1]

3 Suboptimal Strategies

The circumnavigation problem is computationally interesting because of several properties that make it anathema to conventional pathfinding algorithms: the start and end points are the same place, we have a minimum distance quota, and we must operate within the highly irregular patterns of international flight schedules. Before discussing algorithms that work, therefore, I will briefly describe some that do not.

First, to appease the dynamic programming people, there is the **"Ribbon Algorithm"**, which tries to find a route conforming as closely as possible to a great circle divided dyadically into segments. For example, when setting the home airport to London Heathrow and the start time to 6am, the Ribbon Algorithm does the following:

- Defines two segments of the journey: one from London to Auckland (approximate antipodes), and a second one from there back to London;
- Identifies that no well-timed London–Auckland flight exists, so splits this leg into two halves (London–Tokyo and Tokyo–Auckland);
- Identifies that no well-timed London–Tokyo flight exists, so splits this leg into two halves (London–Moscow and Moscow–Tokyo);
- Identifies that no well-timed London–Moscow flight exists, so splits this leg into two halves (London–Copenhagen and Copenhagen–Moscow);
- Finds a London–Copenhagen flight but not a subsequent Copenhagen–Moscow one;
- Hits several different recursion/iteration depth limits and starts returning to previous layers, during which it tries routing through other Scandinavian airports to Moscow, then from London to several other airports in/near Siberia (all of these attempts fail).[2]

...and so on.

The Ribbon Algorithm implements the dynamic programmer's dream that if we just do enough recursion, desirable behaviour will naturally emerge as a result. It is the single least successful piece of software I have ever built. Across a wide variety of parameter adjustments and test cases, this strategy has never once returned a solution. (This is in part because I have designed this algorithm to give up if it has failed to find anything after about thirty minutes, because I have better things to be doing than this.) The Ribbon Algorithm tries and fails to send me through bizarre and unhelpful routes with no comprehension of actual

[1] And frankly, if you have one of those, this whole problem becomes trivial.

[2] It should be noted that the start date I used to test every algorithm in this paper was the 1st of August 2022; at this point, commercial aviation between Russia and most of Europe was non-existent. The Ribbon Algorithm does not understand this.

flight availability, and it has to do several thousand expensive trigonometric calculations at every pass in order to identify the "optimal" airports at which to place a transfer.

We shall leave the world of dynamic programming behind, and try a slightly more canonical approach. When first presented with this problem, I tried to find solutions by hand using what I will call **"Clarkson's Algorithm"** [4], which simply tries to move with the greatest speed possible. We maintain a list of partial routes, sorted by a Speed quotient equal to the total distance covered divided by the time elapsed; at every pass, we look at the fastest route in the list, check all possible subsequent flights departing in the next three hours, and sort the new extended routes back into the list. When we find a route that constitutes a valid circumnavigation, we immediately return it and halt.

Clarkson's Algorithm has the opposite problem to the Ribbon Algorithm; it cares a great deal about flight schedules, but has no sense of direction whatsoever. To ensure it can vaguely make progress, we can hardcode it to only allow eastbound flights while ignoring westbound ones, but this still results in a program that runs around blindly until it happens to stumble into a legal solution. It has a tendency to get stuck in local minima; when setting London as the home airport, for instance, it immediately identifies a very fast and well-timed route to Vancouver via Istanbul and Singapore, but then zigzags around the Americas in a panic trying to build up the remaining 13,000 km (Vancouver–London is only 7,500 km directly). It tries several routes that fail due to a distance deficit, eventually finding a solution that routes through Mexico City, Bogotá and Madrid, returning to London after an inefficient 59 hours and 15 minutes.

The hard eastbound requirement can also cause problems. If setting the home airport to São Paulo, for instance, the algorithm finds zero well-timed trans-Atlantic flights; every allowable route therefore becomes a dead end, and the program crashes without returning anything.

4 Optimal Strategies

Astute readers may observe that Clarkson's Algorithm is vaguely reminiscent of conventional pathfinding algorithms like Dijkstra and A*: we are maintaining a list of partially explored routes sorted by some metric, and at each pass we expand our search from the 'best' route in that list. In A*, the metric in question is $G(n) + H(n)$, where $G(n)$ is the cost of getting to node n, and $H(n)$ is a heuristic providing a lower bound on the further cost of getting from n to the target. It is possible to translate this metric into the language of our problem, as follows:

$$C = t + \frac{\max(d_Q - d, d_{DH})}{v}$$

where C is the cost function to be minimised, t is the time elapsed in the partial route, v is 900 km/h (the approximate cruising speed of a jet aircraft), d is the distance covered by the partial route, d_Q is the distance quota (36,700 km), and d_{DH} is the *directed distance* to the home airport, calculated as:

$$d_{DH} = \begin{cases} d_Q - d_H & |\lambda| < 180^\circ \\ d_H & |\lambda| \geq 180^\circ \end{cases}$$

where d_H is the great-arc distance to the home airport, and λ is the net longitude covered by the route.

The above equations are self-explanatory.

If we design an A*-like pathfinding algorithm using the above cost function, we get what I will call the **O* Algorithm**.[3] This approach performs very well: for London, it produces a solution taking 52:00, a substantial improvement over the 59:15 route found by Clarkson's Algorithm.

Strangely, though, this is not the best possible solution. It is well known that A* search is always guaranteed to return an optimal solution provided the heuristic function is well-designed (i.e. $G(n) + H(n)$ never decreases when we extend a route); however, this does not appear to be the case for O*. I conjecture that this is because the heuristic function is dependent not only on the airport and elapsed time, but also on the specific route taken to get there (which it inherently must be, due to the distance quota requirement); I am, however, not good enough at theoretical computational analysis to prove this.

To find the best possible solution, we will use an algorithm that works similarly to O* but in reverse (which I have naturally called the ***O Algorithm**). This algorithm takes in an acceptable window of time in which we expect our circumnavigation to conclude, and finds a route that finishes somewhere within that window — or if no such route exists, returns a failure message.[4] *O finds all flights that land at the home airport within the accepted window, and searches backward from there using the following modified cost function:

$$C = \frac{t_E \max(d_Q - d, d_{DH})}{vt}$$

where t is the time elapsed as of the departure of the earliest flight in the partial route, t_E is the time elapsed as of the final flight's arrival, and all other variables are the same as above (but backwards, where applicable).

Again, the motivation behind this function should be obvious to the astute reader.

This algorithm works well, in that it if a solution exists within the acceptable finishing window, *O is guaranteed to find it. However, it relies on us selecting a good finishing window in advance of running the program — if the window is too late, we will get suboptimal solutions, and if it is too early, we will get no solution.

We can therefore use the output of the previous O* search to inform the input to the *O search, in a combined approach imaginatively named the **O**O Algorithm**. This works as follows:

1. Run one pass of O*. Assume it finds a solution finishing in T minutes.
2. Run one pass of *O, setting the finishing window to start at $T - 120$ minutes and end at $T - 1$ minutes.
3. If this pass of *O returns a solution, set T to be the finishing time of *that* solution and go back to step 2.
4. If no solution is returned, we know that the most recently returned solution must be the optimal one. Output this as the final result.

For London, this approach eventually produces a solution finishing in 51:40, a 20-minute improvement over plain O*.

For the benefit(?) of the fine people at Carnegie Mellon University, I will give an example by demonstrating what the O**O algorithm does when the home airport is set to to Pittsburgh International. The first pass of the O* stage produces a result finishing in 55:11. The second pass runs the *O algorithm, looking for solutions taking between 53:10 and 55:10; it quickly finds one taking 55:06, a five-minute improvement. The third pass runs *O again, this time looking for a finishing time between 53:05 and 55:05; it produces a failure

[3] So named because the letter O resembles the circle shape in which we are traversing the Earth.

[4] i.e. crashes with a "list index out of range" exception.

message, so we can confirm that the best possible circumnavigation route for Pittsburgh is the following:

Flight	Departs		Arrives	
NZ 6719	PIT Pittsburgh	06:49, 1 Aug	IAH Houston	08:44, 1 Aug
WN 1394	IAH Houston	10:10, 1 Aug	BNA Nashville	12:00, 1 Aug
WN 1904	BNA Nashville	12:30, 1 Aug	LAX Los Angeles	14:45, 1 Aug
OS 82	LAX Los Angeles	15:15, 1 Aug	VIE Vienna	11:50, 2 Aug
NH 6326	VIE Vienna	13:30, 2 Aug	NRT Tokyo	08:50, 3 Aug
AA 8420	NRT Tokyo	10:55, 3 Aug	DFW Dallas	08:40, 3 Aug
AA 2667	DFW Dallas	09:25, 3 Aug	PIT Pittsburgh	13:06, 3 Aug

5 Non-Algorithmic Approaches

The O**O Algorithm, while very successful in principle, has a significant flaw: it is very slow. In the above example for Pittsburgh, my code took 45 minutes to produce the listed route, and a further 45 to verify that it was the fastest option possible (though this is partly because I'm running it on a fairly old and mediocre laptop). There is one final stress test against which we can measure these algorithms: logistics geeks on Discord.

I presented the Pittsburgh circumnavigation problem to a group of friends, and gave them an hour and a half to find the best solution possible using more traditional route-finding tools (Google Flights and a spreadsheet). The results are not strictly comparable, because unlike my code, these people are not working with schedules that are three and a half years out of date. We can, however, compare how long it takes them to find a solution similar or superior to O**O's 55:06.

It took 40 minutes for someone to find a route finishing in 53:24.

6 Conclusion

Aviation nerds 1–0 compsci nerds.

References

[1] Fédération Aéronautique Internationale. *FAI Sporting Code, Section 2 – Aeroplanes*. 2016. URL: https://web.archive.org/web/20160211204923/http://www.fai.org/downloads/gac/SC2. (accessed: 2026-02-28).

[2] Sara Bonner. "The fastest man in the atmosphere". In: *The Times* (12 January 1980), p. 3. URL: https://archive.org/details/NewsUK1980UKEnglish/Jan%2012%201980%2C%20The%20Times%2C%20%2360522%2C%20UK%20%28en%29/page/n1/mode/2up. (accessed: 2026-02-28).

[3] Sam Denby et al. *We Raced To Circumnavigate The Globe In 100 Hours*. 2022. URL: https://www.youtube.com/watch?v=Gta43oOV4Ag. (accessed: 2026-02-28).

[4] Wikipedia. *Jeremy Clarkson*. URL: https://en.wikipedia.org/wiki/Jeremy_Clarkson. (accessed: 2026-02-28).